

How LLMs Work: Tokens, Embeddings, and Transformers

11 min read · Apr 23, 2026



Abdul Moiz

Follow



Listen



Share

Series: LLM Engineering from the Ground Up | Article 1 of 10

Abdul Moiz();

LLM ENGINEERING

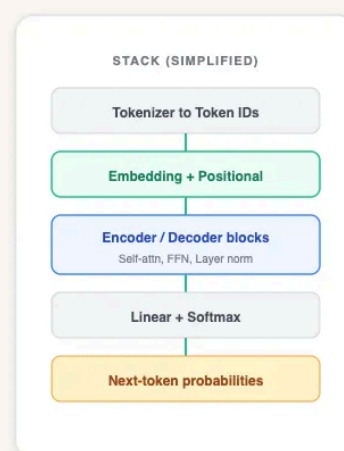
LLM Engineering from ground up - Basic

Tokens, embeddings, and the Transformer - how modern LLMs represent language.

Article 1



Abdul Moiz
Software Engineer



Hey everyone !! back again with one of the series and probably be completing it on time :) here we are. Thoda late, but better than never. So I jumped into LLM engineering properly this time. I took **Ed Donner** course on Udemy, read papers, read articles, filled up notebooks, and now I am here to break it all down for you in a way that actually makes sense. No jargon without explanation. No skipping the “why.”

This is article one of a series of five to seven. We are building this up piece by piece, and by the end of this series you will have a solid mental model of how large language models actually work under the hood. Each article builds on the one before it, so it is worth reading them in order.

Let's get into it.

Why It Matters

Everything in this article is foundational. Tokens, embeddings, and the Transformer architecture sit under every modern LLM you have ever used. ChatGPT, Claude, Gemini, all of them. Once you understand these three things, the rest starts to click naturally.

Think of it like learning how a car engine works before you start tuning one. You do not have to know it to drive, but if you want to build something on top of these models, this is where you start.

Tokens

Here is the first thing to get straight. A model does not read words the way you do. It does not read characters either. It reads tokens.

A token is a chunk of text. It could be a full word, part of a word, or even just punctuation. The exact split depends on the tokenizer, but the goal is always the same: break text into manageable pieces that the model can learn to work with.

Take the word "playing." To you, that is one word. To a tokenizer, that is two tokens: "play" and "ing." The word "unhappiness" might split into "un" and "happiness." The word "cat" stays as one token because it is short and common enough.

Why not just use whole words? Because the vocabulary would be enormous. There are millions of unique words across all languages, including typos, names, and technical terms. By splitting words into subwords, you get a compact vocabulary of maybe 50,000 to 100,000 tokens that can still handle rare words by combining smaller pieces.

Example

Here is a real example. Take the sentence:

"I love eating sushi in Tokyo."

After tokenization it might look like this:

["I", "love", "eat", "ing", "su", "shi", "in", "Tokyo", "."]

And each token gets assigned a Token ID from a fixed vocabulary lookup table:

[40, 1842, 4483, 278, 7143, 12490, 287, 23361, 13]

That list of numbers is what the model actually receives as input. Not English. Numbers. This changes how you think about what a model reads entirely.

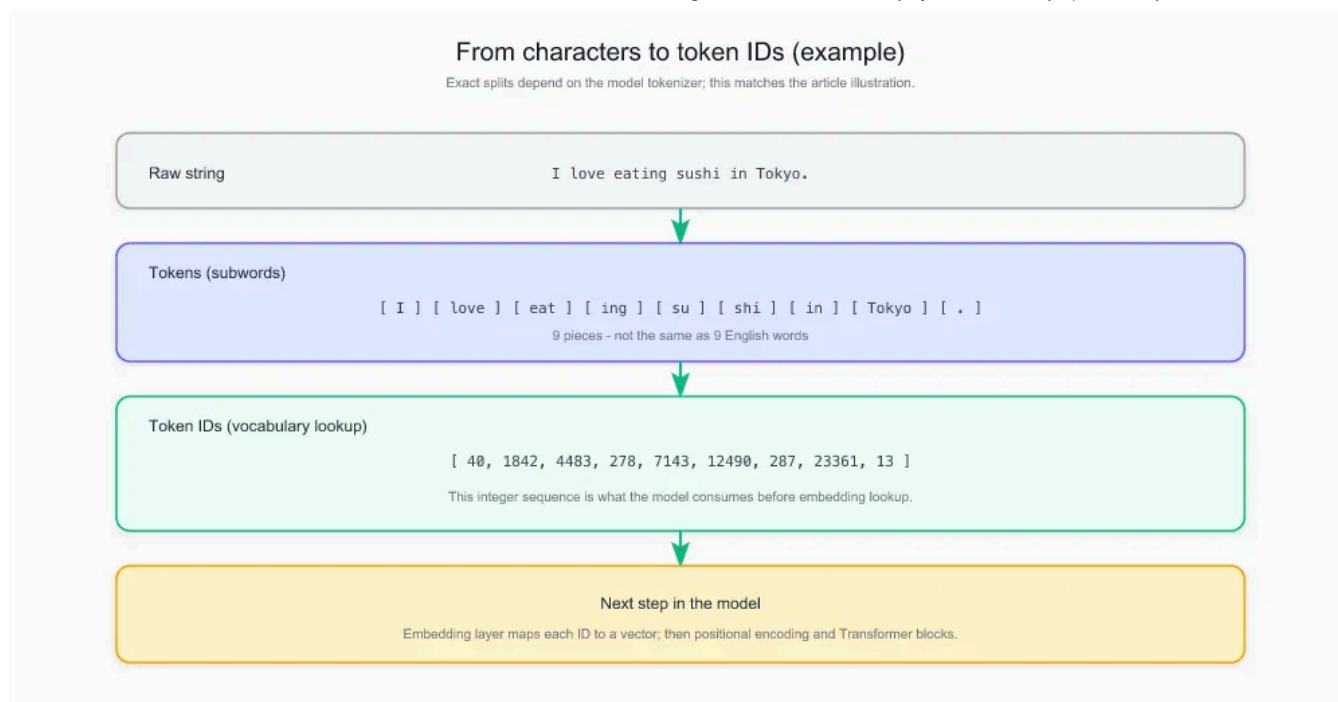
Tokenization

Tokenization is the process of converting raw text into those token IDs. It happens before the model sees anything.

Every model has its own tokenizer trained alongside it. GPT-4 uses a tokenizer called tiktoken. BERT uses WordPiece. LLaMA uses SentencePiece. They each split text slightly differently, which is why the same sentence might produce a different number of tokens depending on which model you use.

This also explains why models charge by tokens rather than by words or characters. A sentence with many rare or long words produces more tokens and costs more to process than a simple sentence with common short words.

You can test this yourself. OpenAI has a free tokenizer tool at platform.openai.com/tokenizer. Type in any sentence and watch how it splits. The color-coded output is genuinely eye-opening the first time you see it.



Embeddings

Now you have a list of Token IDs. The problem is that a number like 40 has no inherent relationship to 1842. They are just integers. The model has no idea that “cat” and “kitten” are related, or that “Paris” and “France” share something in common.

This is where embeddings come in.

An embedding is a way of representing a token as a point in a high-dimensional space, where tokens with similar meanings end up close to each other. Each token gets mapped to a list of numbers called a vector. After training, tokens that appear in similar contexts end up with similar vectors. “Dog” and “puppy” will be close together in that space. “Dog” and “algorithm” will be far apart.

The entire point is to encode meaning into geometry.

Static Embeddings

The two classic approaches to word embeddings are Word2Vec and GloVe.

Word2Vec was introduced by Google in 2013. The idea is simple. You train a small neural network on a huge corpus of text and give it one of two tasks. Either predict a word given its surrounding words, or predict the surrounding words given a single word.

For example, given the sentence “The cat sat on the ___”, the model tries to predict “mat.” It does this millions of times across billions of sentences. The weights the network learns to make those predictions become the embeddings.

After training on something like Wikipedia or a large book corpus, the model figures out that “Paris” and “France” have the same relationship as “Berlin” and “Germany.” That relationship gets encoded in the geometry of the vector space.

Example

The famous example of this is vector arithmetic. You can literally compute:

$$\text{vector}(\text{“king”}) - \text{vector}(\text{“man”}) + \text{vector}(\text{“woman”}) = \text{vector}(\text{“queen”})$$

The model learned gender and royalty as directions in space, without anyone ever explicitly teaching it those concepts.

GloVe, from Stanford, takes a different approach. Instead of training on local context windows, it looks at global word co-occurrence statistics across the entire corpus. It counts how often “ice” appears near “cold” across all documents, and how often “steam” appears near “hot.” It then finds vectors that best explain those co-occurrence counts. The results are similar in quality to Word2Vec, but the training method is fundamentally different.

Both of these are called Static Embeddings, and here is the core problem with them.

Every token gets exactly one vector, no matter where it appears. The word “bank” gets one fixed vector forever. But look at these two sentences:

“I went to the bank to withdraw cash.”

“We sat by the river bank and watched the water.”

Both use the word “bank” but mean completely different things. A static embedding averages out all those contexts into one blurry point in space. The model cannot tell which meaning you intended.

This is a real limitation. Language is deeply contextual. You need embeddings that shift based on surrounding words.

Contextual Embeddings

Contextual embeddings solve exactly this. Instead of one fixed vector per word, you compute the embedding dynamically based on the full context of the sentence.

Get Abdul Moiz's stories in your inbox

Join Medium for free to get updates from this writer.

Subscribe



Remember me for faster sign in

Models like BERT pioneered this. When BERT processes “bank” in a sentence about finance, it produces a different vector than when it processes “bank” in a sentence about a river. The vector is shaped by everything around it.

Example

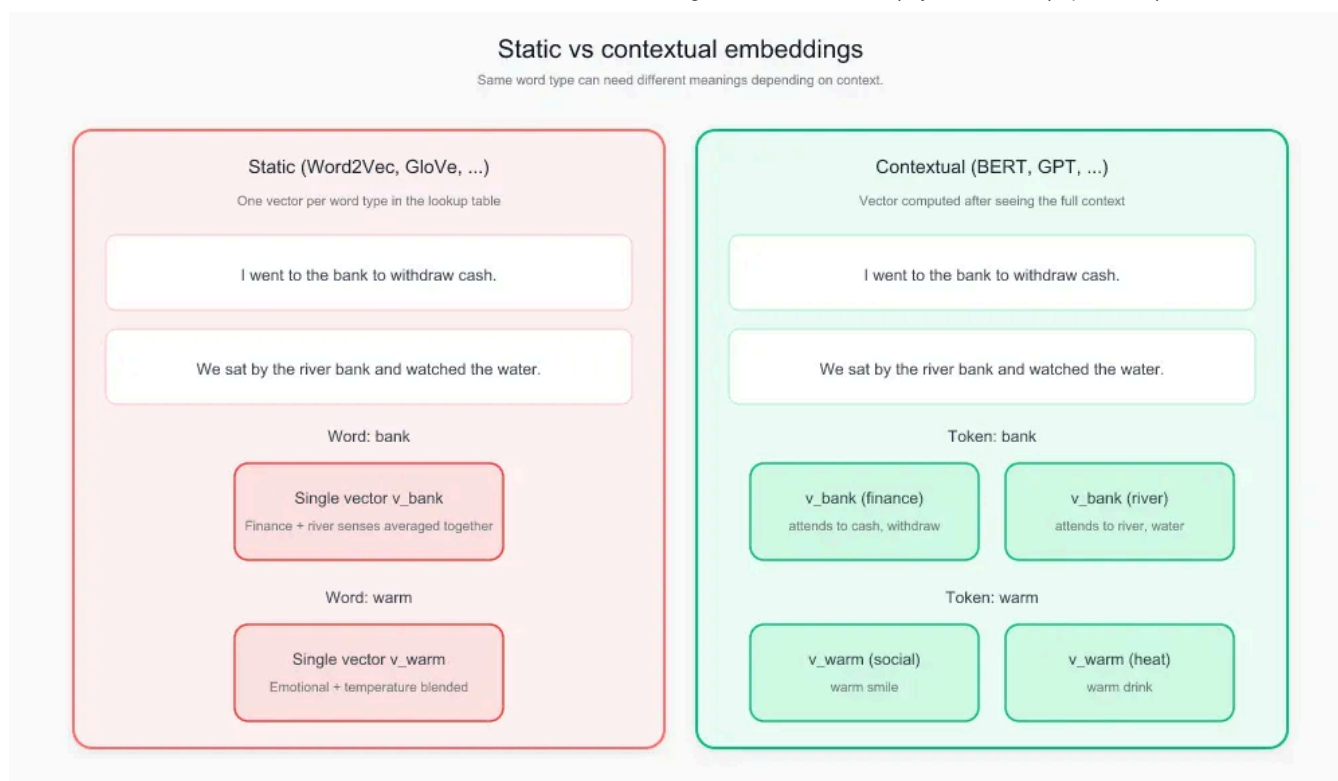
Here is another example. Take these two sentences:

“He gave her a warm smile.”

“She poured herself a warm drink.”

The word “warm” means something different in each. In the first sentence it describes an emotional gesture. In the second it describes temperature. A contextual embedding model produces two different vectors for “warm” here. A static model gives you the same vector both times.

This is only possible because of the Transformer architecture, which processes all the words in a sentence simultaneously and figures out how each one relates to every other. That process is what makes contextual embeddings work.



Embedding Vectors

Let's be concrete for a moment. An embedding vector in a model like GPT-2 has 768 dimensions. In GPT-3, it jumps to 12,288 dimensions. Each dimension is a learned floating point number.

You will never directly interpret what any single dimension means. Dimension 142 does not neatly correspond to “animal” or “positive sentiment.” The meaning is distributed across all dimensions together. What matters is the distance and direction between vectors in that space.

Similarity is measured using cosine similarity, which measures the angle between two vectors. A score of 1.0 means identical direction. A score of 0.0 means no relationship. So “puppy” and “dog” might score 0.91, while “puppy” and “democracy” might score 0.04.

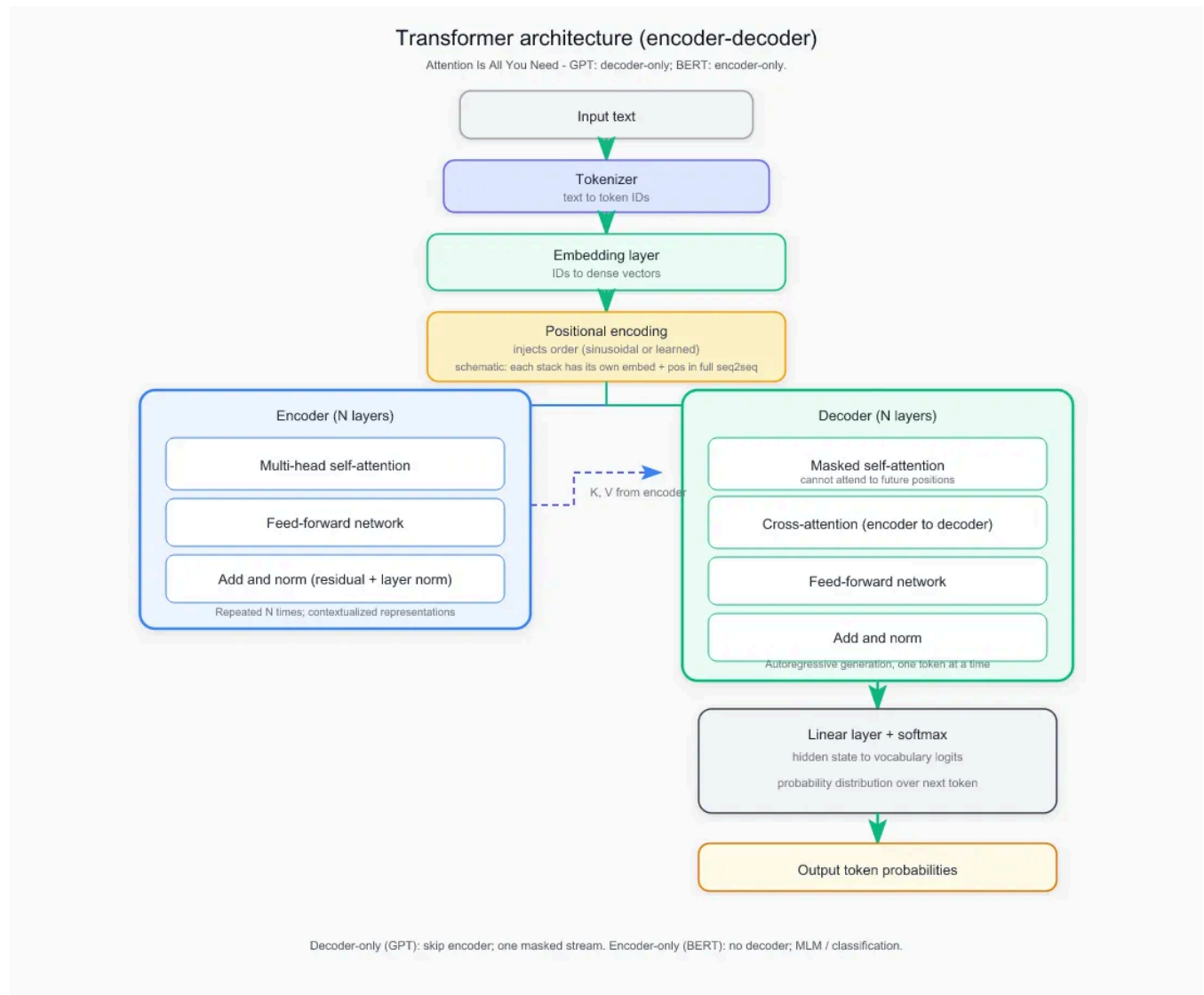
This vector space is where all the semantic knowledge of a language model lives. Before any Transformer logic runs, your tokens get converted into these vectors. That is the raw material the model reasons with.

Transformer

Everything above was about representing language as numbers. Now we get to the architecture that actually processes those numbers.

The Transformer was introduced in a 2017 paper called “Attention Is All You Need” by researchers at Google. It replaced the dominant architecture at the time, recurrent neural networks, and became the foundation of every major LLM today.

The Transformer has two main components: an Encoder and a Decoder. Here is the full flow:



Positional Encoding

The Embedding Layer takes your token IDs and converts them to vectors. You already know this part.

Positional Encoding gets added on top. The Transformer processes all tokens at the same time in parallel, which makes it fast. But that means it has no natural sense of word order.

Without positional information, “Dog bites man” and “Man bites dog” would look identical. Positional encodings inject order by adding a unique mathematical

pattern to each position's vector. Position 1 gets a slightly different signal than position 2, position 3, and so on. The model learns to use these signals to understand sequence.

Self-Attention

Self-Attention is the heart of the Transformer. For every single token, the model asks: which other tokens in this sequence are most relevant to understanding me right now?

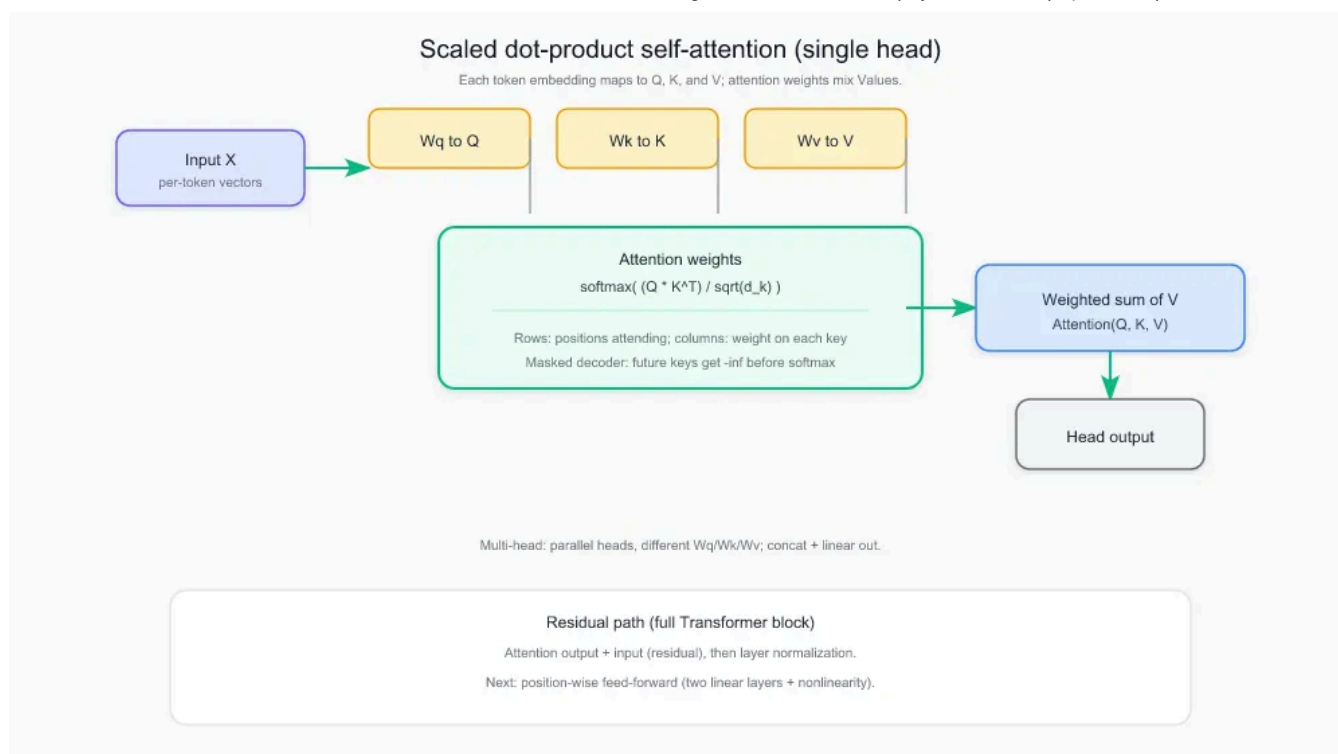
Take this sentence: "The trophy did not fit in the suitcase because it was too big."

What does "it" refer to? The trophy, not the suitcase. You know this because "too big" fits the trophy. Self-attention is the mechanism that figures this out. The token "it" learns to attend strongly to "trophy" after processing the surrounding context.

Here is how it works at a basic level. For each token, the model computes three things: a Query, a Key, and a Value. Think of it like a search engine. The Query is what a token is looking for. The Keys are what each other token is advertising about itself. The model computes a score between the Query and every Key, then creates a weighted average of the Values based on those scores.

The result is a new, richer vector for each token that has absorbed relevant information from the rest of the sequence.

Multi-Head Attention runs this process several times in parallel with different learned projections each time. One head might focus on grammatical relationships. Another might track coreference. Another might pick up on semantic similarity. The outputs all get combined at the end.



Encoder vs Decoder

The Encoder reads the full input and builds a rich contextual representation. Every token's vector gets updated based on its relationship to every other token. You end up with a sequence of deeply informed vectors.

Models like BERT use only the Encoder. They are designed for understanding tasks like text classification, question answering, and named entity recognition, not for generating text.

The Decoder generates output one token at a time. At each step, it attends to what it has already generated through Masked Self-Attention (masked so it cannot peek at future tokens it has not generated yet), and it also attends to the Encoder's output through Cross-Attention. This is how the Decoder stays grounded in what the input actually said.

GPT-style models use only the Decoder. They generate text left to right, predicting the most likely next token at each step. GPT-4, Claude, LLaMA, Mistral: all Decoder-only.

Here is what actually happens when you send a message to ChatGPT. Your text gets tokenized. Those tokens get converted to embedding vectors. Those vectors pass through dozens of Decoder blocks. At each step the model produces a probability distribution over all 50,000-plus tokens in its vocabulary. The token with the highest

probability gets selected, appended to the sequence, and the process repeats until the model hits a stop signal.

That loop, one token at a time, is how your entire chat response gets generated.

Parameters

When people say a model has 7 billion or 70 billion parameters, they mean the total count of learnable numbers inside the model. Every attention weight, every embedding value, every matrix in every feed-forward layer is a parameter.

During training, the model processes billions of tokens and adjusts these parameters using gradient descent to get better at predicting the next token. After training, those parameters are frozen. They now encode everything the model knows about language, facts, reasoning, and style.

To put scale into perspective, **GPT-3 has 175 billion parameters**. Each one is a 32-bit floating point number. That alone is around 700 gigabytes of raw data before any compression or quantization.

When a model responds to you, it is not searching a database. It is running your input through billions of matrix multiplications using those frozen parameters. The answer comes entirely from patterns absorbed during training.

More parameters generally means more capacity. But training data quality, training duration, and architecture decisions all matter just as much. A well-trained smaller model often outperforms a poorly trained larger one.

What's Next

This series is five to seven articles long. In the next pieces we will go deeper into how attention actually gets computed, how models are trained from scratch through pretraining and fine-tuning, what RAG is and how it connects models to external knowledge, and eventually how you build production systems on top of these models as an engineer.

Each article builds on the last. Save this one. Come back to it when later articles reference these concepts.

. . .

That's enough for today. If your brain feels a bit overloaded, that's normal. This stuff takes time and a few rereads to really click.

Drop your questions in the comments and I'll get back to you. You can also reach out directly at abdulmoiz3140@gmail.com.

If you want to go deeper one-on-one, I'm available on [Topmate](#) where we can break things down at your pace.

At [DevWeekends](#), we focus on building a strong engineering mindset and solid fundamentals. If you're learning and want a structured path with accountability, reach out at info@devweekends.com and explore the [resources](#) on the [DevWeekends](#) website.

See you in the next one.

LLM

Transformers

Embedding

Tokenization

Ai Engineering

[Follow](#)

Written by Abdul Moiz

191 followers · 19 following

A developer always struggling to code , part time reader . occasional writer—here when inspiration strikes. Let's learn and connect through bugs & books!"

Responses (1)



Write a response

What are your thoughts?



Abdullahghulamnabi

May 16



Great article, @Abdul Moiz. Loved the way the content was explained, so easy but thorough. Will love to read upcoming ones.



[Reply](#)

